PROJECT REPORT

Multi-Vendor E-Commerce Marketplace

An Amazon-Style Platform for Vendors, Customers, and Marketplace Commission

Full-stack capstone project

GitHub Repository	**https://github.com/madhan023-mn/Capstone_Projects_2**
Project Type	Full-stack web application
Frontend	React 18, Vite, Tailwind CSS
Backend	Node.js, Express.js, databases and supporting services

1. Executive Summary

The Multi-Vendor E-Commerce Marketplace is a web application intended to bring multiple independent vendors and customers together on one online shopping platform. Vendors can manage product listings and orders, customers can browse products and make purchases, and administrators can manage users, vendors, orders, product categories, and promotions. The platform is designed around the marketplace business model, in which the platform earns commission from vendor sales.

The repository contains a React frontend and a Node.js/Express backend. The backend includes modules for authentication, products, categories, vendors, carts, orders, reviews, coupons, uploads, and administration. The implementation also includes integrations or configuration for MongoDB, Redis, Elasticsearch, payment providers such as Stripe and Razorpay, and optional S3-backed uploads. The report describes the repository structure and intended capabilities; it does not claim that every integration has been independently tested in production.

2. Problem Statement

Traditional single-seller online stores are primarily designed for one business to manage the complete product catalogue. A multi-vendor marketplace must support many sellers, separate vendor inventories and order views, customer shopping journeys, marketplace-wide administration, and calculation of platform commission. Without a coordinated system, managing these activities can become difficult and error-prone.

This project aims to centralize these workflows in one application while providing distinct interfaces and permissions for customers, vendors, and administrators.

3. Objectives

- Create a shared marketplace where multiple vendors can list and manage products.
- Allow customers to register, sign in, browse products, and manage a shopping cart.
- Support checkout and order tracking through dedicated customer pages and backend routes.
- Provide vendor workflows for product management, order handling, and earnings.
- Provide administrators with controls for users, vendors, categories, coupons, and orders.
- Maintain data through backend models and database configuration.
- Support marketplace commission tracking and vendor-related order records.
- Add security, validation, caching, search, and testing capabilities where configured.

4. Main User Roles

Role	Main responsibilities	Relevant screens/modules
Customer	Browse products, cart, checkout, orders, reviews, profile and wishlist	Home, catalog, product detail, cart, checkout, orders, profile, wishlist
Vendor	Apply to sell, manage products, process orders, review earnings	Vendor application, dashboard, products, product form, orders, earnings
Administrator	Oversee users, vendors, categories, coupons and orders	Admin dashboard, users, vendors, categories, coupons, orders

5. Core Features

5.1 Customer shopping

Customer-facing pages include a home page, product catalogue, product details, login and registration, cart, checkout, order history, order details, profile management, and wishlist. The frontend includes cart, authentication, and currency contexts to share related state across components.

5.2 Vendor management

Vendor pages support the vendor application flow, dashboard, product listing and editing, order management, and earnings views. Backend vendor models and controllers help separate vendor-specific information and operations from the general customer experience.

5.3 Administration

The admin interface includes dashboard, users, vendors, categories, coupons, and orders. These features are intended to help platform operators manage the marketplace and review its activity.

5.4 Product catalogue and discovery

Product and category modules support catalogue-related workflows. Elasticsearch is configured as an optional search service, while the repository also defines a Redis configuration for caching or related data-access support.

5.5 Cart, orders, and vendor order records

Cart and order modules handle the shopping journey from selected products through order creation and order views. The data models include Order and VendorOrder, which support representing a customer's order alongside vendor-specific order handling.

5.6 Reviews and promotions

Review and coupon modules provide routes and controllers for product feedback and promotional discounts. Coupon administration is available through a dedicated admin page.

5.7 Marketplace commission

The repository includes CommissionLedger and VendorProfile models and a configurable DEFAULT_COMMISSION_PERCENT setting, shown as 10 percent in the example environment file. The effective commission calculation, settlement rules, refunds, and accounting should be confirmed and tested against the implementation before using real payments.

5.8 Uploads and payments

The backend includes file-upload routes and configuration for optional AWS S3 storage with a local uploads fallback. Dependencies and environment examples include Stripe and Razorpay in test mode. Payment-provider configuration is not proof of live payment processing; test-mode credentials and webhook verification must be configured correctly before use.

6. Technology Stack

Layer	Technology	Purpose
Frontend	React 18, Vite	Interactive browser UI and development/build tooling
Routing and HTTP	React Router, Axios	Page navigation and API communication
Styling	Tailwind CSS, PostCSS	Responsive UI styling
Backend	Node.js, Express.js	REST API and application server
Authentication	JWT, bcryptjs	Token-based authentication and password hashing
Primary configured database	MongoDB/Mongoose; SQLite support is also present	Persist marketplace entities and support local/test or alternate data paths
Cache	Redis	Optional caching or shared fast data access
Search	Elasticsearch	Optional catalogue search service
Payments	Stripe and Razorpay SDKs	Payment-provider integrations, configured for test mode in example settings
File storage	Local uploads / optional AWS S3	Product and other uploaded media
Validation and security	Zod, validator, Helmet, CORS, express-rate-limit	Input validation and HTTP/API safeguards
Testing	Jest, Supertest	Backend unit/API test support

7. System Architecture

The project is organized into frontend and backend applications. The frontend uses React pages, reusable components, shared contexts, and an Axios API client. It calls REST endpoints implemented by the Express backend.

- Frontend: customer, vendor, and administrator pages; navigation components; product cards; authentication and cart contexts; API client.
- Backend routes: API paths for authentication, categories, vendors, products, cart, orders, administration, coupons, reviews, uploads, and health checks.

- Controllers: business logic for handling requests and coordinating data operations.
- Models: data structures for users, products, categories, carts, orders, vendor profiles, vendor orders, commission ledger entries, coupons, addresses, and reviews.
- Middleware: authentication/authorization and centralized error handling.
- Infrastructure: database connection, Redis setup, Elasticsearch setup, local upload support, and environment-based configuration.

Typical shopping flow: a customer browses the catalogue in the React application; the frontend sends API requests to Express; the backend validates the request and identity, applies business logic, reads or updates marketplace data, and returns a response for the UI. During checkout, the application creates order-related records and can integrate a configured payment provider. Vendor and administrator pages use their corresponding endpoints to manage operations.

8. Data Model Overview

The model directory shows the principal business entities. Their exact field definitions and relationships should be read from the source files when producing an implementation-level database diagram.

- User: account details and role-related identity.
- Product and Category: product listings and catalogue classification.
- VendorProfile and VendorOrder: vendor information and vendor-specific order workflows.
- Cart and Address: selected items and customer delivery information.
- Order: customer purchase and order lifecycle.
- CommissionLedger: records associated with marketplace commission.
- Coupon and Review: discount campaigns and product feedback.

9. Security and Reliability

- JWT and bcryptjs dependencies support authentication and password hashing; access-control rules should be verified for every role and protected endpoint.
- Helmet, request rate limiting, validation libraries, and centralized error handling are included in the backend configuration.
- CORS origins should be restricted to explicit trusted production domains. The current server code contains a permissive fallback that approves other origins, so this should be tightened before production.
- Keep real credentials, JWT secrets, database URLs, payment secrets, and cloud storage keys out of source control. Use secret management or protected environment variables.
- Verify payment webhook signatures, idempotency, amount/currency validation, and order state transitions before processing real transactions.
- Confirm authorization at the backend for vendor-owned products and orders; frontend route protection alone is insufficient.
- Configure backups, logging, monitoring, and recovery procedures for database and dependent services.

These are source-based review observations and recommended safeguards, not a formal security audit.

10. Testing

The backend package defines a Jest test command and includes authentication test files under backend/tests. Supertest is available for HTTP endpoint tests. The repository's test files should be executed in a configured environment to confirm actual pass/fail status.

- Test registration, login, invalid credentials, token verification, and expired/invalid tokens.
- Test customer, vendor, and admin permissions on relevant API endpoints.
- Test product CRUD operations, vendor ownership, category filtering, cart quantities, and stock constraints.
- Test checkout validation, order creation, commission calculations, retries, refunds, and payment webhook handling.
- Test coupon validity dates, usage limits, review validation, upload limits, and unsupported file types.
- Test behavior when Redis or Elasticsearch is unavailable and confirm the intended fallback paths.

11. Local Setup and Execution

The repository includes separate frontend and backend package manifests. A typical development setup is:

1. Install a compatible Node.js LTS version and npm.
2. Clone https://github.com/madhan023-mn/Capstone_Projects_2.git and open the project folder.
3. In backend, install dependencies with npm install and copy .env.example to .env.
4. Set appropriate database and JWT environment variables. Use test credentials for payment integrations and local or development services for Redis/Elasticsearch as needed.
5. In one terminal, run the backend using npm run dev from the backend directory.
6. In another terminal, install frontend dependencies with npm install inside frontend, then run npm run dev.
7. Open the local URL displayed by Vite, commonly http://localhost:5173, and verify the frontend API configuration points to the running backend.
8. Run backend tests using npm test from the backend directory after the required environment is configured.

Exact commands, dependency compatibility, database mode, environment values, and any deployment configuration should be checked against the current source before running the application. Never paste real secrets into a report or commit them to GitHub.

12. Limitations and Future Enhancements

- Expand the README with complete installation instructions, architecture diagrams, environment-variable documentation, and demo screenshots.
- Add or document a complete API reference with request/response examples and role requirements.
- Harden CORS and thoroughly audit authentication, authorization, file uploads, and payment webhooks.

- Make database strategy explicit across MongoDB, SQLite, and PostgreSQL dependencies/configuration, and document which mode is supported for local, test, and production environments.
- Add end-to-end tests for customer checkout and vendor/admin workflows.
- Validate stock consistency and transactional behavior when multiple customers purchase the same product.
- Document commission settlement, refunds, cancellations, and vendor payout rules.
- Add monitoring, structured logging, backups, retry policies, and service health checks.
- Measure catalogue/search performance and cache invalidation behavior under realistic workloads.

13. Conclusion

The Multi-Vendor E-Commerce Marketplace is a full-stack project designed to support multiple sellers within a shared online shopping platform. Its repository contains distinct customer, vendor, and admin interfaces, a modular Express API, marketplace-related data models, and configuration for search, caching, uploads, and payment providers. The project demonstrates relevant full-stack concepts such as role-based workflows, REST API design, product and order management, and marketplace commission tracking. Before a production launch, the main priorities are verifying all flows through tests, tightening access controls and CORS, and confirming payment, commission, database, and deployment behavior.

14. References

1. GitHub repository: https://github.com/madhan023-mn/Capstone_Projects_2

2. README: https://github.com/madhan023-mn/Capstone_Projects_2/blob/main/README.md

3. Backend server entry point: https://github.com/madhan023-mn/Capstone_Projects_2/blob/main/backend/server.js

4. Backend package manifest: https://github.com/madhan023-mn/Capstone_Projects_2/blob/main/backend/package.json

5. Frontend package manifest: https://github.com/madhan023-mn/Capstone_Projects_2/blob/main/frontend/package.json

Note: The repository README provides only a brief project description, so this report also refers to the visible source tree, package manifests, environment example, server configuration, controllers, routes, pages, and models. Runtime behavior and third-party integrations have not been independently verified.